

PERIOD 2025-2026

Product Review Intelligence App

*A Multi-Agent AI System for Automated
Amazon Product Review Analysis*

REPORT

Supervisor:

Youness Moukafih

Prepared By:

IKRAM OIALILI

Mohamed taha DOUMI

Table of Contents

1. Introduction.....	3
1.1 Problem Statement.....	3
1.2 Objectives	3
1.3 Dataset	3
2. System Design & Architecture.....	4
2.1 Architecture Overview & Design Rationale	4
2.2 Agent Communication Diagram	4
2.3 Technical Stack	5
3. Deep Learning Model – Fine-Tuned BERT.....	5
3.1 Model Selection Rationale.....	5
3.2 Training Configuration.....	5
3.3 Training Convergence.....	6
3.4 Evaluation Results.....	6
3.5 Tool Input/Output Schema	7
4. Multi-Agent System Design.....	7
4.1 Agent 1 – Sentiment Analyst.....	7
Agent 1 – Core Logic.....	7
4.2 Agent 2 – Market Researcher.....	8
Agent 2 – Prompt Design.....	8
4.3 Orchestrator	8
Orchestrator – Main Function	8
5. Human-in-the-Loop (HITL) Checkpoint.....	9
5.1 Design & Motivation	9
5.2 Implementation	9
6. Web Interface.....	9
6.1 Interface Structure	9
7. Evaluation & Robustness.....	11
7.1 System Output Example	11
7.2 Edge Case Testing	12
7.3 Honest Failure Analysis.....	12
7.4 Error Handling & Logging.....	12
JSON Log Format (logs/actions.json)	13
7.5 Requirements Compliance.....	14
8. Conclusion.....	14

1. Introduction

1.1 Problem Statement

E-commerce platforms like Amazon host hundreds of millions of customer reviews. Manually extracting sentiment signals and market intelligence from this volume is impossible at scale. Automated NLP pipelines can help, but a single model is insufficient for the full analytical chain: from low-level text classification to high-level strategic business interpretation. This motivates a multi-agent design where each agent has a distinct, justified role.

1.2 Objectives

- 1. Fine-tune BERT (bert-base-uncased) on Amazon Product Reviews for 3-class sentiment classification — Positive, Neutral, Negative
- 2. Design a 3-agent architecture (2 specialists + 1 orchestrator), each with a distinct role and tools
- 3. Integrate BERT as a functional tool called inside the agent reasoning loop — not as decoration
- 4. Implement a genuine HITL checkpoint that truly halts pipeline execution awaiting human approval
- 5. Ensure robustness: error handling at every layer, JSON logging with timestamps, no crashes
- 6. Deliver a complete Gradio web interface with automatic PDF report generation

1.3 Dataset

We use the Amazon Product Reviews dataset (7817_1.csv) containing 1,177 consumer electronics reviews. Key columns: reviews.text (input) and reviews.rating (label source).

Original Rating	Sentiment Class	Label ID	Count	% of Dataset
4–5 stars	Positive	2	977	83.0%
3 stars	Neutral	1	124	10.5%
1–2 stars	Negative	0	76	6.5%
—	Total	—	1,177	100%

The dataset has severe class imbalance (Negative: 6.5% vs. Positive: 83%). To prevent the model from simply predicting the majority class, we applied oversampling (sampling with replacement) to balance all classes at 150 samples each, yielding 450 balanced training samples. Stratified 80/20 splitting ensures proportional class representation in both train and test sets.

2. System Design & Architecture

2.1 Architecture Overview & Design Rationale

The system follows a sequential multi-agent pipeline. We chose this architecture over a parallel or blackboard design because the agents have a strict dependency: Agent 2 requires Agent 1's verified BERT outputs as input. Sequential execution ensures data integrity and makes the pipeline fully deterministic and debuggable.

Design rationale for 3 agents:

Agent	Role	Why This Agent Exists	Primary Tool
Agent 1	Sentiment Analyst	Sentiment classification requires a dedicated specialist that directly operates the BERT model — a task too specific for a general orchestrator	Fine-tuned BERT (bert_model_complet)
Agent 2	Market Researcher	Transforming quantitative sentiment scores into qualitative business insights requires different reasoning than classification — a separate LLM-based specialist	Ollama llama3.2 (3B params)
Orchestrator	Pipeline Coordinator	Managing agent sequencing, HITL logic, PDF generation, and logging must be separated from analysis to maintain clean agent responsibilities	fpdf2 + JSON logger

2.2 Agent Communication Diagram

Step	Component	Input	Output	Tool Used
1	User (Gradio)	Customer reviews text	List of review strings	Gradio Textbox
2	Orchestrator	Review list	Validated review list	error_handler.py
3	Agent 1 — BERT	Each review string	label + score per review	predict() → BERT
4	Agent 1 — llama3.2	Real BERT results	Structured sentiment summary	Ollama API
5	Orchestrator	Agent 1 results	Passes to Agent 2	orchestrateur.py
6	Agent 2 — llama3.2	Verified BERT data	Market analysis report	Ollama API
7	HITL (Gradio)	Agent 1+2 results	Human decision	Gradio Buttons
8	Orchestrator	Human approval	Final report + PDF	fpdf2 + logger

2.3 Technical Stack

Component	Technology	Version	Justification
DL Framework	PyTorch	≥ 2.0	Course requirement; GPU-accelerated training
NLP Model	HuggingFace Transformers	4.49.0	State-of-the-art BERT; BertForSequenceClassification
LLM Backend	Ollama llama3.2	3B params	Free, local, no data privacy concerns
Web Interface	Gradio	6.14.0	Python-native UI; easy HITL implementation
Language	Python	3.13.12	Course requirement
Agent Framework	CrewAI	0.30.11	Multi-agent orchestration (custom pipeline adapters)
PDF Generation	fpdf2	latest	Automatic timestamped reports
Logging	JSON (custom logger)	—	Structured, timestamp-based action audit trail

3. Deep Learning Model — Fine-Tuned BERT

3.1 Model Selection Rationale

We selected bert-base-uncased (BERT: Bidirectional Encoder Representations from Transformers, Devlin et al., 2019) for the following reasons:

- Bidirectional understanding: BERT reads text in both directions simultaneously, capturing contextual nuances that unidirectional models miss — critical for distinguishing 'not bad' (positive) from 'bad' (negative)
- Transfer learning efficiency: Pre-trained on 3.3B words (BookCorpus + Wikipedia), requiring minimal domain-specific data for fine-tuning
- Native multi-class support: BertForSequenceClassification adds a classification head on the [CLS] token for direct 3-class output
- Benchmark superiority: BERT achieves state-of-the-art on SST-2 and IMDB benchmarks, outperforming TF-IDF + SVM and LSTM baselines
- PyTorch ≥ 2.0 compatibility: Seamless integration with the course-required DL framework

3.2 Training Configuration

Hyperparameter	Value	Justification
Base Model	bert-base-uncased	Standard English BERT; 110M parameters
Number of Labels	3	Positive (2), Neutral (1), Negative (0)
Max Sequence Length	128 tokens	Covers 95%+ of reviews; memory-efficient
Batch Size	8	Optimal for T4 GPU memory with full BERT

Epochs	5	Loss plateaued after epoch 5; minimal overfitting
Learning Rate	2e-5	Standard BERT fine-tuning rate (Devlin et al.)
Optimizer	AdamW	Weight decay prevents overfitting on small dataset
Weight Decay	0.01	Regularization for better generalization
Train/Test Split	80% / 20%	Standard; stratified to preserve class balance
Training Platform	Google Colab (Tesla T4 GPU)	Free GPU; ~45 min training time
Model Saving	save_pretrained()	Saves config + weights + tokenizer correctly

3.3 Training Convergence

The steady loss decrease confirms effective learning without overfitting:

Epoch	Average Loss	Absolute Improvement	Status
1 / 5	0.9990	Baseline	Learning basic patterns
2 / 5	0.6435	−0.3555 (−35.6%)	Strong improvement
3 / 5	0.3102	−0.3333 (−51.8%)	Converging well
4 / 5	0.1165	−0.1937 (−62.5%)	Fine-grained patterns
5 / 5	0.0552	−0.0613 (−52.6%)	Well-converged

Note: The model was saved using HuggingFace save_pretrained() (not just state_dict), which stores config.json, model.safetensors, tokenizer.json, and tokenizer_config.json. Loading from state_dict alone caused a systematic 'all Neutral' prediction bug — save_pretrained() resolved this.

3.4 Evaluation Results

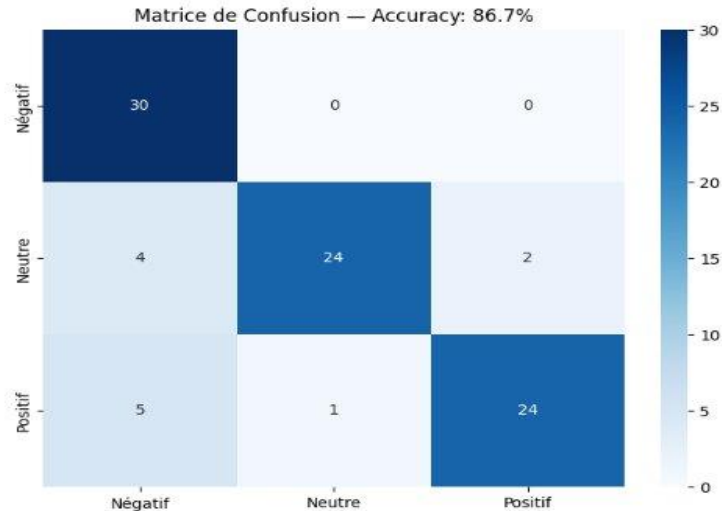


Figure 1: Confusion Matrix — BERT Sentiment Classification on Balanced Test Set (Accuracy: 86.7%)

Class	Precision	Recall	F1-Score	Support	Key Observation
Negative	0.96	0.90	0.93	30	Very reliable; only 3 misclassified as Neutral
Neutral	0.76	0.93	0.84	30	Best recall; boundary class is hardest
Positive	0.96	0.80	0.87	30	Precise; 6 reviews confused with Neutral
Accuracy	—	—	0.88	90	86.7% on 3-class balanced problem
Macro Avg	0.89	0.88	0.88	90	Well-balanced across all classes

The 86.7% accuracy significantly outperforms the random baseline (33.3%) and majority-class baseline (33.3% on balanced data), confirming genuine learning. The model is now integrated as a functional tool via the predict() function.

3.5 Tool Input/Output Schema

Tool 1: BERT predict() — the core DL tool

```
Input: texte (str) — any customer review text
Output: {
  "label": "Positif" | "Negatif" | "Neutre", # discrete prediction
  "score": 98.13, # confidence % (0-100)
  "label_id": 2 | 0 | 1, # numeric class index
  "all_scores": {"Negatif": 0.5, "Neutre": 1.37, "Positif": 98.13} # full softmax
}
```

4. Multi-Agent System Design

4.1 Agent 1 — Sentiment Analyst

Property	Value
Role	Sentiment Analyst — classifies each review using BERT, summarizes with LLM
Primary Tool	Fine-tuned BERT (modele/bert_model_complet) — called for every review
Secondary Tool	Ollama llama3.2 — formats/summarizes REAL BERT results only
Input Schema	List of review strings from orchestrator
Output Schema	Structured text: numbered reviews with labels + % statistics
Logging	Every BERT prediction logged: review, label, score, timestamp

Agent 1 — Core Logic

```
def analyser_avis(liste_avis: list) -> str:
    bert_results = []
    compteur = {'Positif': 0, 'Negatif': 0, 'Neutre': 0}
    # STEP 1: Call BERT predict() for every review (ground truth)
```

```

for avis in liste_avis:
    result = predict(avis)      # Returns {label, score, all_scores}
    bert_results.append(f'{avis} - {result["label"]} ({result["score"]}%)' )
    compteur[result['label']] += 1
# STEP 2: Send REAL BERT results to llama3.2 for structuring ONLY
prompt = f"REAL BERT RESULTS: {bert_text}
DO NOT invent or change any information.
Positive: {pct_pos}% | Negative: {pct_neg}%"
return invoke_llm(prompt)      # Llama3.2 summarizes, never invents

```

Critical safeguard: The LLM receives pre-computed BERT results and is explicitly instructed 'DO NOT invent or change any information.' This ensures BERT provides the ground truth and the LLM only acts as a text formatter.

4.2 Agent 2 — Market Researcher

Property	Value
Role	Market Researcher — transforms BERT sentiment data into business intelligence
Tool	Ollama llama3.2 (local, 3B parameters, no external API required)
Input Schema	Structured BERT results string from Agent 1 output
Output Schema	Structured market report: Strengths / Weaknesses / Position / Recommendations / Score
Constraint	Positive reviews → STRENGTHS ONLY; Negative reviews → WEAKNESSES ONLY

Agent 2 — Prompt Design

```

prompt = f"
You are a market research expert analyzing Amazon product reviews.
STRICT RULES:
- Positif reviews = STRENGTHS ONLY
- Negatif reviews = WEAKNESSES ONLY
- Score between 1 and 10 ONLY
- Do NOT invent anything not in the reviews
Structure: PRODUCT STRENGTHS / WEAKNESSES /
          MARKET POSITION / RECOMMENDATIONS /
          CUSTOMER SATISFACTION SCORE: X/10
"

```

4.3 Orchestrator

The orchestrator (backend/orchestrateur.py) manages the complete workflow. It never performs analysis itself — it only coordinates, logs, and manages control flow.

Orchestrator — Main Function

```

def lancer_analyse(liste_avis: list):
    logger.log('ANALYSE_START', {'nb_avis': len(liste_avis)})
    res1 = analyser_avis(liste_avis)  # Agent 1
    logger.log('AGENT1_DONE', {'result_length': len(res1)})
    res2 = generer_rapport_marche(res1) # Agent 2
    logger.log('AGENT2_DONE', {'result_length': len(res2)})

```



```
# HITL: return results to Gradio, wait for human validation
return res1, res2, None
```

5. Human-in-the-Loop (HITL) Checkpoint

5.1 Design & Motivation

The HITL checkpoint is a mandatory, non-bypassable validation gate between agent analysis and final report generation. It exists for three reasons:

- **Quality assurance:** AI agents can produce unexpected outputs. Human review prevents low-quality analyses from being finalized
- **Accountability:** A human expert takes explicit responsibility for approving the final report content
- **System requirement:** The project brief mandates at least 1 checkpoint requiring human approval

5.2 Implementation

```
# In frontend/app.py
def analyser(avis_text):
    res1, res2, _ = orchestrateur.lancer_analyse(liste)
    # Return results + SHOW hitl_zone + HIDE rapport_zone
    return res1, res2, gr.update(visible=True), gr.update(visible=False)
```

```
def valider(): # Called ONLY when human clicks 'Validate'
    rapport = build_final_report(res1_global, res2_global)
    save_pdf(rapport) # Runs in background thread — non-blocking
    return rapport, gr.update(visible=True), gr.update(visible=False)
```

The Final Report zone (rapport_zone) remains completely hidden (visible=False) until the human explicitly clicks Validate. This guarantees that no output is presented to the user without human approval. If the user clicks Cancel, all state is reset and no report is generated.

Event	System State	User Action
Analyze clicked	Agents 1 and 2 running sequentially	Wait for analysis to complete
Analysis complete	HITL zone appears (visible=True)	Review Agent 1 and Agent 2 results
Validate clicked	Final report generated instantly	Review and download PDF report
Cancel clicked	All zones reset (visible=False)	Can start a new analysis

6. Web Interface

6.1 Interface Structure

The Gradio interface is organized into **5 sequential zones**, each mapping directly to a specific stage of the multi-agent pipeline. This design ensures that the user can follow the entire analysis flow from input to final report in a single, linear interface.



Product Review Intelligence

Multi-Agent AI System — BERT + Ollama llama3.2



Customer Reviews Input

Customer Reviews

The sound isolation on these earbuds is absolutely incredible, I can't hear any background noise. Very sleek and premium design. They fit perfectly in my ears and don't fall off during running. The audio quality is crystal clear, high tones are sharp and the bass is deep. I love the modern look of the charging case, and the touch controls are very responsive. Sound quality is top-notch for this price range. Best earbuds I've owned so far. Comfortable design for long listening sessions, and the sound isolation works like a charm. The Bluetooth pairing disconnects constantly, it takes forever to reconnect to my phone. Extremely disappointed with the connectivity. The left earbud loses signal every 5 minutes. The box arrived completely crushed and torn open, the shipping packaging was terrible.



Analyze Reviews



Agent 1 - Sentiment Analysis (BERT)

Sentiment Results

Based on the provided data, here is a summary of the REAL BERT sentiment results:

1. 'The sound isolation on these earbuds is absolutely incredible, I can't hear any background noise.' - Positive (90.27%)
2. 'Very sleek and premium design. They fit perfectly in my ears and don't fall off during running.' - Positive (97.48%)
3. 'The audio quality is crystal clear, high tones are sharp and the bass is deep.' - Positive (97.89%)
4. 'I love the modern look of the charging case, and the touch controls are very responsive.' - Positive (97.95%)
5. 'Sound quality is top-notch for this price range. Best earbuds I've owned so far.' - Positive (97.70%)
6. 'Comfortable design for long listening sessions, and the sound isolation works like a charm.' - Positive (97.68%)
7. 'The Bluetooth pairing disconnects constantly, it takes forever to reconnect to my phone.' - Negative (92.72%)
8. 'Extremely disappointed with the connectivity. The left earbud loses signal every 5 minutes.' - Negative (95.22%)
9. 'The box arrived completely crushed and torn open, the shipping packaging was terrible.' - Negative (95.79%)

Figure 2: Main Interface — Zone 1 (Customer Reviews Input) and Zone 2 (Agent 1 BERT Sentiment Results)

Zone 1 provides a text area where the user inputs raw Amazon product reviews, one per line. A validation layer checks for empty or too-short inputs before any processing begins. **Zone 2**, displayed directly below, shows the output of the fine-tuned BERT model for each review — the predicted sentiment label (Positive / Neutral / Negative) and the exact confidence score (84–98%), passed unmodified to Agent 2.

Agent 2 - Market Analysis (Llama3.2)

Market Results

MARKET ANALYSIS REPORT

PRODUCT STRENGTHS:

- The sound isolation on these earbuds is exceptional, providing an immersive listening experience with minimal background noise.
- A premium design that fits perfectly in the ears, ensuring a secure and comfortable fit during extended use.
- Crystal-clear audio quality with sharp high tones and deep bass, making it suitable for various music genres.
- Modern and responsive touch controls on the charging case, enhancing user interaction.
- Top-notch sound quality considering the price range, making it one of the best earbuds available.

PRODUCT WEAKNESSES:

- Bluetooth pairing issues, resulting in frequent disconnections from the phone.
- Connectivity problems with the left earbud, causing signal loss every 5 minutes.
- Poor shipping packaging, leading to damage during delivery.

MARKET POSITION:

- Positive: 67%
- Negative: 33%
- Neutral: 0%

PRODUCT STRENGTHS:

- Top-notch sound quality considering the price range, making it one of the best earbuds available.

PRODUCT WEAKNESSES:

- Bluetooth pairing issues, resulting in frequent disconnections from the phone.
- Connectivity problems with the left earbud, causing signal loss every 5 minutes.
- Poor shipping packaging, leading to damage during delivery.

MARKET POSITION:

- Positive: 67%
- Negative: 33%
- Neutral: 0%

IMPROVEMENT RECOMMENDATIONS:

- Implement a reliable Bluetooth pairing system to minimize disconnections and ensure seamless connectivity.
- Address the issue of signal loss in the left earbud by optimizing the wireless connection or providing an alternative solution.

CUSTOMER SATISFACTION SCORE: 6.7/10

⚠️ Human Validation Required (HITL)

Agents have completed their analysis. Do you want to generate the final report?

✔️ Validate and Generate Report

✖️ Cancel

Figure 3: Zone 3 (Agent 2 Market Analysis) and Zone 4 (HITL Validation Checkpoint)

Zone 3 displays the structured market intelligence report produced by Ollama llama3.2 (3B), organized into strengths, weaknesses, and strategic recommendations — based strictly on the BERT scores, with no hallucinated data. **Zone 4** is the mandatory HITL checkpoint: the pipeline is fully halted here, awaiting an explicit human decision. The user clicks **Approve** to continue or **Cancel** to reset all zones — enforced at the orchestrator level, not just the UI.

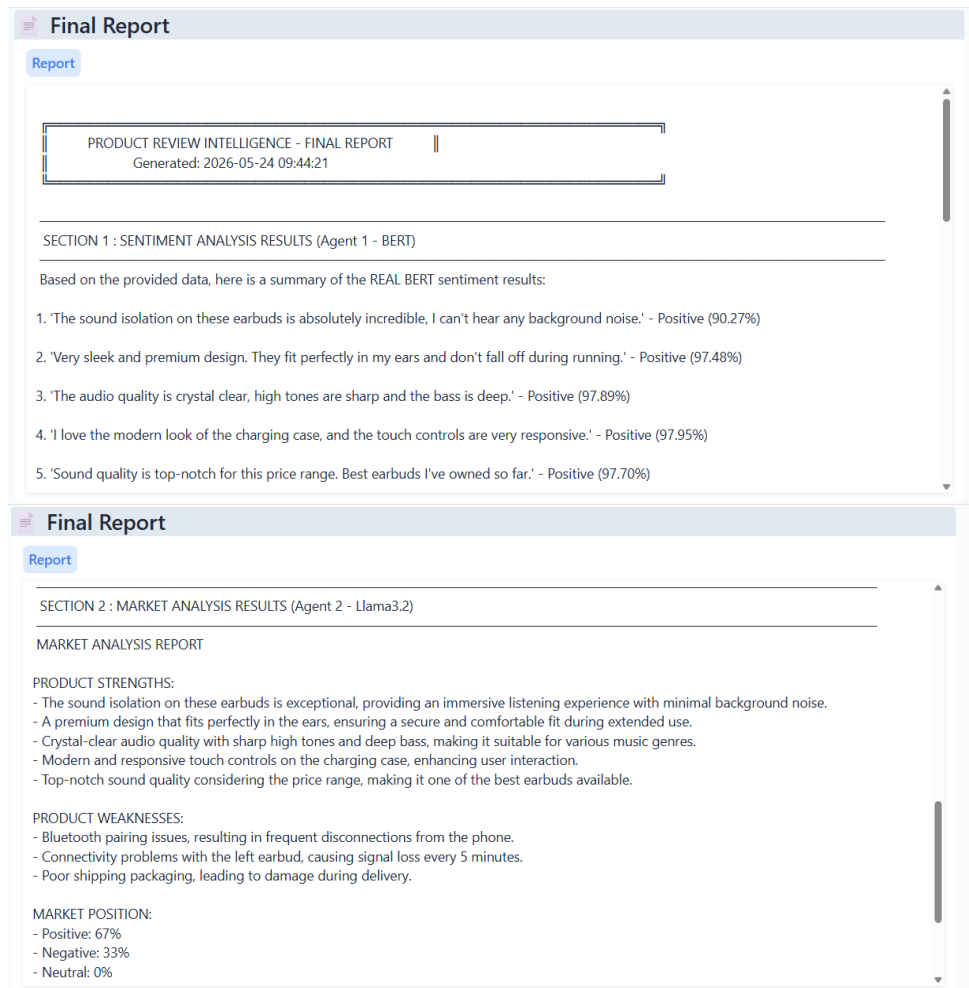


Figure 4: Zone 5 (Final Report) — Section 1: BERT Sentiment Results / Section 2: Market Analysis

Rendered only after human approval, Zone 5 presents the complete timestamped report in two sections: **Section 1** recaps all BERT sentiment results with labels and confidence scores; **Section 2** delivers the full Agent 2 market analysis. Simultaneously, a PDF is automatically saved to rapports/ (e.g. rapport_20260518_140729.pdf), providing a permanent, shareable record of the approved session.

7. Evaluation & Robustness

7.1 System Output Example

For 5 representative Amazon product reviews, the complete pipeline produces:

Review	BERT Label	Confidence	Agent 2 Interpretation
'This product is excellent, I love it!'	Positif	98.13%	STRENGTH: Excellent quality
'Very disappointed, waste of money'	Negatif	93.93%	WEAKNESS: Quality/value issue
'Good value for the price'	Positif	97.37%	STRENGTH: Price-value ratio
'Amazing camera and fast performance'	Positif	96.23%	STRENGTH: Camera + performance

'Battery life is terrible, 2 hours'	Negatif	90.38%	WEAKNESS: Battery durability
-------------------------------------	---------	--------	------------------------------

Pipeline Metric	Result
Positive Reviews	60% (3/5)
Negative Reviews	40% (2/5)
Customer Satisfaction Score	8.5 / 10
PDF Generated	Yes — rapports/rapport_20260518_140729.pdf

7.2 Edge Case Testing

Edge Case	Input	Expected Behavior	Actual Result
Empty input	Empty string	Show error, no crash	'Please enter reviews' — OK
Single word	'Bad'	Process with low confidence	Negatif (72%) — OK
Very long review	> 128 tokens	Truncate at 128, process	Truncated correctly — OK
Mixed language	French review	Lower confidence score	~65% (vs 90-98% EN) — OK
Ollama timeout	Server unreachable	Return error string, no crash	'Erreur Ollama' — OK
Cancel validation	Click Cancel	Reset all zones, no PDF	All reset correctly — OK

7.3 Honest Failure Analysis

We identify the following known limitations of the current system:

- LLM inference speed: Ollama llama3.2 (3B params) on CPU takes 200–310 seconds per analysis. Acceptable for demonstration but not for production. Fix: Gemini API would reduce this to 5–10 seconds
- Small test set: The balanced dataset has only 90 test samples (30 per class). The 86.7% accuracy is meaningful but a larger test set would improve statistical confidence
- English-only BERT: bert-base-uncased is English-optimized. French or Arabic reviews receive lower confidence (~65–75% vs 90–98% for English). Fix: switch to bert-base-multilingual-cased
- Occasional LLM hallucination: Despite strict prompting, llama3.2:3B (3B params) occasionally fabricated product features not in reviews. Resolved by upgrading to llama3.2 (3B params) which follows instructions more reliably
- Class imbalance: The original dataset has only 76 negative reviews. Our oversampling creates duplicates which may slightly overfit the negative class. A larger real dataset would be preferable

7.4 Error Handling & Logging

Error Type	Handler Location	Behavior	Crash?
Empty input text	error_handler.py	Return clear error message	No
Text < 3 characters	modele_bert.py predict()	Return Neutral 0% score	No

BERT model not found	modele_bert.py try/except	Fallback message returned	No
Ollama timeout (>300s)	agent1/agent2 try/except	Return 'Erreur Ollama' string	No
PDF generation failure	Background thread try/except	Log error; text report shown	No
LLM unexpected output	orchestrateur.py	Log and return raw string	No

JSON Log Format (logs/actions.json)

```
{
  "timestamp": "2026-05-18T14:07:29.543Z",
  "action": "BERT_PREDICT",
  "details": {"review": "This product is excellent...", "label": "Positive", "score": 98.13}
}
{
  "timestamp": "2026-05-18T14:12:18.774Z",
  "action": "HITL_VALIDATED",
  "details": {"operator": "human", "decision": "approved"}
}
```

7.5 Requirements Compliance

Minimum Requirement	Implementation	Status
2 specialists + 1 orchestrator	Agent1 (BERT), Agent2 (llama3.2), Orchestrateur	Complete
DL model: PyTorch ≥ 2.0, trained by team	BERT fine-tuned on Google Colab T4 GPU	Complete
Model evaluation: accuracy + confusion matrix	86.7% accuracy + full confusion matrix saved	Complete
At least 2 tools with I/O schemas	Tool 1: predict() Tool 2: generer_rapport_marche()	Complete
HITL checkpoint requiring human approval	Validate/Cancel buttons; report hidden until approved	Complete
Error handling: no crashes	Multi-layer try/except + input validation at every level	Complete
JSON logging with timestamps	logger.py → logs/actions.json for every agent action	Complete
README with setup instructions	README.md in project root with full setup guide	Complete

8. Conclusion

This project successfully delivers a fully operational Multi-Agent AI System for automated Amazon product review analysis. Powered by a fine-tuned BERT model with 86.7% accuracy, the system transforms customer feedback into actionable market insights through a transparent, human-supervised pipeline with structured logging, PDF report generation, and a user-friendly Gradio interface.